

```
!apt-get install -y ffmpeg -qq
!pip install -q librosa soundfile numpy scipy matplotlib requests python-docx
```

```
import requests, zipfile, io, os

# <-- REPLACE THIS with your ZIP file URL (public URL) -->
zip_url = "https://github.com/karoldvl/ESC-50/archive/refs/heads/master.zip"
out_dir = "q4_files"
os.makedirs(out_dir, exist_ok=True)

print("Downloading zip from:", zip_url)
r = requests.get(zip_url)
r.raise_for_status()
z = zipfile.ZipFile(io.BytesIO(r.content))
z.extractall(out_dir)
print("Extracted to:", out_dir)
print("Files:")
for root, _, files in os.walk(out_dir):
    for f in files:
        print("-", os.path.join(root, f))

- q4_files/ESC-50-master/audio/1-62850-A-38.wav
- q4_files/ESC-50-master/audio/4-67358-A-42.wav
- q4_files/ESC-50-master/audio/1-17124-A-43.wav
- q4_files/ESC-50-master/audio/3-127890-B-9.wav
- q4_files/ESC-50-master/audio/5-200334-A-1.wav
- q4_files/ESC-50-master/audio/2-70344-A-33.wav
- q4_files/ESC-50-master/audio/5-188365-A-36.wav
- q4_files/ESC-50-master/audio/3-162786-A-13.wav
- q4_files/ESC-50-master/audio/1-46040-A-14.wav
- q4_files/ESC-50-master/audio/2-37806-A-40.wav
- q4_files/ESC-50-master/audio/5-261464-A-23.wav
- q4_files/ESC-50-master/audio/3-134699-B-16.wav
- q4_files/ESC-50-master/audio/3-172179-A-31.wav
- q4_files/ESC-50-master/audio/3-144510-A-30.wav
- q4_files/ESC-50-master/audio/3-119120-B-48.wav
- q4_files/ESC-50-master/audio/5-207811-B-35.wav
- q4_files/ESC-50-master/audio/4-144468-A-27.wav
- q4_files/ESC-50-master/audio/3-149465-A-22.wav
- q4_files/ESC-50-master/audio/1-32579-A-29.wav
- q4_files/ESC-50-master/audio/3-118656-A-41.wav
- q4_files/ESC-50-master/audio/5-233787-A-7.wav
- q4_files/ESC-50-master/audio/1-46272-A-12.wav
- q4_files/ESC-50-master/audio/3-108677-A-18.wav
- q4_files/ESC-50-master/audio/4-188293-B-15.wav
- q4_files/ESC-50-master/audio/2-109231-A-9.wav
- q4_files/ESC-50-master/audio/2-66637-B-20.wav
- q4_files/ESC-50-master/audio/5-185516-A-27.wav
- q4_files/ESC-50-master/audio/3-128160-A-44.wav
- q4_files/ESC-50-master/audio/1-17808-A-12.wav
- q4_files/ESC-50-master/audio/2-107228-B-44.wav
- q4_files/ESC-50-master/audio/5-103422-A-2.wav
- q4_files/ESC-50-master/audio/5-103418-A-2.wav
- q4_files/ESC-50-master/audio/4-191687-A-0.wav
- q4_files/ESC-50-master/audio/5-259640-A-29.wav
- q4_files/ESC-50-master/audio/2-173607-A-39.wav
- q4_files/ESC-50-master/audio/1-34094-A-5.wav
- q4_files/ESC-50-master/audio/1-30709-B-23.wav
- q4_files/ESC-50-master/audio/5-156999-E-19.wav
- q4_files/ESC-50-master/audio/1-79236-A-36.wav
- q4_files/ESC-50-master/audio/1-47273-A-21.wav
- q4_files/ESC-50-master/audio/5-220955-A-40.wav
- q4_files/ESC-50-master/audio/1-104089-A-22.wav
- q4_files/ESC-50-master/audio/2-108767-B-9.wav
- q4_files/ESC-50-master/audio/4-147240-B-2.wav
- q4_files/ESC-50-master/audio/4-99644-C-4.wav
- q4_files/ESC-50-master/audio/4-207116-A-23.wav
- q4_files/ESC-50-master/audio/3-117293-A-9.wav
- q4_files/ESC-50-master/audio/1-196660-A-8.wav
- q4_files/ESC-50-master/audio/4-119647-C-48.wav
- q4_files/ESC-50-master/audio/3-155234-A-43.wav
- q4_files/ESC-50-master/audio/3-115387-A-47.wav
- q4_files/ESC-50-master/audio/2-72547-B-14.wav
- q4_files/ESC-50-master/audio/5-215447-A-47.wav
- q4_files/ESC-50-master/audio/1-58923-B-27.wav
- q4_files/ESC-50-master/audio/4-189836-A-22.wav
- q4_files/ESC-50-master/audio/3-152594-A-30.wav
- q4_files/ESC-50-master/.github/stale.yml
- q4_files/ESC-50-master/tests/test_dataset.py
```

```
import glob, librosa
from IPython.display import display, Audio

audio_paths = []
```

```

for ext in ("wav","mp3","flac","ogg","m4a"):
    audio_paths += glob.glob(f"{out_dir}/**/*.{ext}", recursive=True)

if not audio_paths:
    raise FileNotFoundError("No audio files found in extracted ZIP. Check zip_url and contents.")

# Prefer files with "clean" in filename (case-insensitive)
clean_candidates = [p for p in audio_paths if "clean" in os.path.basename(p).lower()]
if clean_candidates:
    clean_path = clean_candidates[0]
else:
    # fallback: choose the longest-duration audio file (assumed to be clean)
    durations = {}
    for p in audio_paths:
        try:
            y, sr = librosa.get_duration(filename=p), None
            durations[p] = y
        except Exception:
            durations[p] = 0
    # get file with largest size as proxy if librosa duration fails
    clean_path = max(audio_paths, key=lambda p: os.path.getsize(p))
print("Selected clean file:", clean_path)

# List noisy files (exclude the clean one)
noisy_files = [p for p in audio_paths if p != clean_path]
print("Detected noisy files:", noisy_files)

# Load clean audio (resample to 22050 mono)
clean_y, sr = librosa.load(clean_path, sr=22050, mono=True)
print("Clean audio length (s):", len(clean_y)/sr)
display(Audio(clean_y, rate=sr))

```

```

/tmp/ipython-input-1273203514.py:20: FutureWarning: get_duration() keyword argument 'filename' has been renamed to 'path'.
This alias will be removed in version 1.0.
  y, sr = librosa.get_duration(filename=p), None
Selected clean file: q4_files/ESC-50-master/audio/3-165856-A-41.wav
Detected noisy files: ['q4_files/ESC-50-master/audio/2-64962-A-15.wav', 'q4_files/ESC-50-master/audio/5-179496-A-15.wav']
Clean audio length (s): 5.0

0:03 / 0:05

```

```

import numpy as np
import matplotlib.pyplot as plt
import scipy.signal as sig
import soundfile as sf
from docx import Document
from docx.shared import Inches

eps = 1e-10

def save_plot_wave_and_fft(y, sr, out_png_path, title_prefix=""):
    # waveform + FFT magnitude plot side by side
    fig, axes = plt.subplots(1,2, figsize=(12,4))
    t = np.arange(len(y))/sr
    axes[0].plot(t, y, linewidth=0.5)
    axes[0].set_title(f"{title_prefix} Waveform")
    axes[0].set_xlabel("Time (s)")
    axes[0].set_xlim(0, t[-1])

    # FFT magnitude
    N = len(y)
    Y = np.fft.rfft(y * np.hanning(N))
    f = np.fft.rfftfreq(N, 1/sr)
    axes[1].semilogy(f, np.abs(Y)+eps, linewidth=0.5)
    axes[1].set_title(f"{title_prefix} FFT Magnitude")
    axes[1].set_xlabel("Frequency (Hz)")
    axes[1].set_xlim(0, sr/2)
    axes[1].grid(True, which='both', alpha=0.2)

    plt.tight_layout()
    plt.savefig(out_png_path, dpi=150)
    plt.close()
    return out_png_path

def normalized_corr(a, b):
    L = min(len(a), len(b))
    a = a[:L]; b = b[:L]
    num = np.sum(a * b)
    den = np.sqrt(np.sum(a*a) * np.sum(b*b)) + eps
    return num/den

```

```

def estimate_noise_peaks(clean, noisy, sr, n_peaks=3, min_freq=50, max_freq=8000):
    # compute magnitude spectra
    N = 2**int(np.ceil(np.log2(max(len(clean), len(noisy)))))
    C = np.abs(np.fft.rfft(clean, n=N))
    Y = np.abs(np.fft.rfft(noisy, n=N))
    noise_mag = np.maximum(0, Y - C)
    freqs = np.fft.rfftfreq(N, 1/sr)
    # limit search range
    idx = np.where((freqs>=min_freq) & (freqs<=max_freq))[0]
    mag_slice = noise_mag[idx]
    # find peaks
    from scipy.signal import find_peaks
    peaks, props = find_peaks(mag_slice, height=np.mean(mag_slice)+np.std(mag_slice))
    peak_freqs = freqs[idx][peaks] if len(peaks)>0 else np.array([])
    # sort by magnitude and return top n_peaks
    if peak_freqs.size == 0:
        return []
    peak_mags = mag_slice[peaks]
    order = np.argsort(peak_mags)[::-1]
    selected = list(peak_freqs[order][:n_peaks])
    return selected

def design_iir_bandstop(sr, f0, bw=10, order=4):
    # design a Butterworth bandstop centered at f0 with bandwidth bw (Hz)
    nyq = sr/2
    low = max((f0 - bw/2)/nyq, 1e-6)
    high = min((f0 + bw/2)/nyq, 0.999999)
    b, a = sig.butter(order, [low, high], btype='bandstop')
    return b, a

def apply_iir_notches(y, sr, freqs_to_notch, bw=30, order=4):
    y_filt = y.copy()
    for f0 in freqs_to_notch:
        b,a = design_iir_bandstop(sr, f0, bw=bw, order=order)
        # use filtfilt for zero-phase
        y_filt = sig.filtfilt(b, a, y_filt)
    return y_filt

def design_fir_bandstop(sr, f0, bw=30, numtaps=1023):
    nyq = sr/2
    low = max((f0 - bw/2)/nyq, 0.0)
    high = min((f0 + bw/2)/nyq, 1.0)
    bands = [0, low, high, 1.0]
    desired = [1, 0, 1]
    # convert normalized to bands in firwin2 format
    # We'll use firwin with pass_zero True via concatenation of two filters (lowpass + highpass)
    # Simpler: design via remez if available; fallback to firwin low/high then convolve - but for simplicity:
    # Use scipy.signal.firwin2 to create bandstop
    import numpy as np
    freqs = [0, low, high, 1.0]
    mags = [1, 0, 1, 1]
    # firwin2 expects freq in 0..1 where 1 is Nyquist
    taps = sig.firwin2(numtaps, freqs, mags, window='hamming')
    return taps

def apply_fir_notches(y, sr, freqs_to_notch, bw=30, numtaps=1023):
    y_filt = y.copy()
    for f0 in freqs_to_notch:
        taps = design_fir_bandstop(sr, f0, bw=bw, numtaps=numtaps)
        y_filt = sig.fftconvolve(y_filt, taps, mode='same')
    return y_filt

def save_wav(y, path, sr):
    # normalize to avoid clipping
    y = y / (np.max(np.abs(y))+eps) * 0.95
    sf.write(path, y.astype(np.float32), sr)

```

```

import librosa
import numpy as np
import matplotlib.pyplot as plt
import scipy.signal as sig
from IPython.display import Audio, display
import pandas as pd

results = []

print("\n=====")
print("Q4 - IIR & FIR RESULTS BELOW")
print("=====\\n")

for noisy_path in noisy_files:
    name = os.path.basename(noisy_path)

```

```

name = os.path.basename(noisy_path)
print("\n-----")
print("Processing:", name)
print("-----")

noisy_y, _ = librosa.load(noisy_path, sr=sr, mono=True)
noisy_y = noisy_y[:len(clean_y)]
L = len(noisy_y)

# -----
# FFT plot
# -----
N = len(noisy_y)
fft_vals = np.abs(np.fft.rfft(noisy_y))
freq = np.fft.rfftfreq(N, 1/sr)

plt.figure(figsize=(10,3))
plt.semilogy(freq, fft_vals)
plt.title(f"FFT Spectrum - {name}")
plt.xlabel("Frequency (Hz)")
plt.ylabel("Magnitude")
plt.grid()
plt.show()

# -----
# Noise peak detection
# -----
clean_fft = np.abs(np.fft.rfft(clean_y[:L]))
noise_fft = fft_vals - clean_fft
peak_idx = np.argmax(noise_fft)
noise_freq = freq[peak_idx]

print("Detected Noise Frequency:", noise_freq, "Hz")

# -----
# IIR NOTCH FILTER
# -----
w0 = noise_freq / (sr/2)
b, a = sig.iirnotch(w0, 30)
iir_out = sig.filtfilt(b, a, noisy_y)

print("IIR Filter Output:")
display(Audio(iir_out, rate=sr))

# Waveform plot
plt.figure(figsize=(10,2))
plt.plot(iir_out)
plt.title(f"IIR Filtered Waveform - {name}")
plt.show()

# correlation
corr_iir = np.sum(clean_y[:L]*iir_out) / (
    (np.sqrt(np.sum(clean_y[:L]**2)*np.sum(iir_out**2))) + 1e-10
)
print("Correlation (Clean vs IIR):", corr_iir)

# -----
# FIR BAND-STOP FILTER
# -----
nyq = sr/2
low = (noise_freq - 40)/nyq
high = (noise_freq + 40)/nyq
taps = sig.firwin2(801, [0, low, high, 1], [1,0,1,1])

fir_out = sig.fftconvolve(noisy_y, taps, mode="same")

print("FIR Filter Output:")
display(Audio(fir_out, rate=sr))

# Waveform plot
plt.figure(figsize=(10,2))
plt.plot(fir_out)
plt.title(f"FIR Filtered Waveform - {name}")
plt.show()

# correlation
corr_fir = np.sum(clean_y[:L]*fir_out) / (
    (np.sqrt(np.sum(clean_y[:L]**2)*np.sum(fir_out**2))) + 1e-10
)
print("Correlation (Clean vs FIR):", corr_fir)

# -----
# Store table entry

```

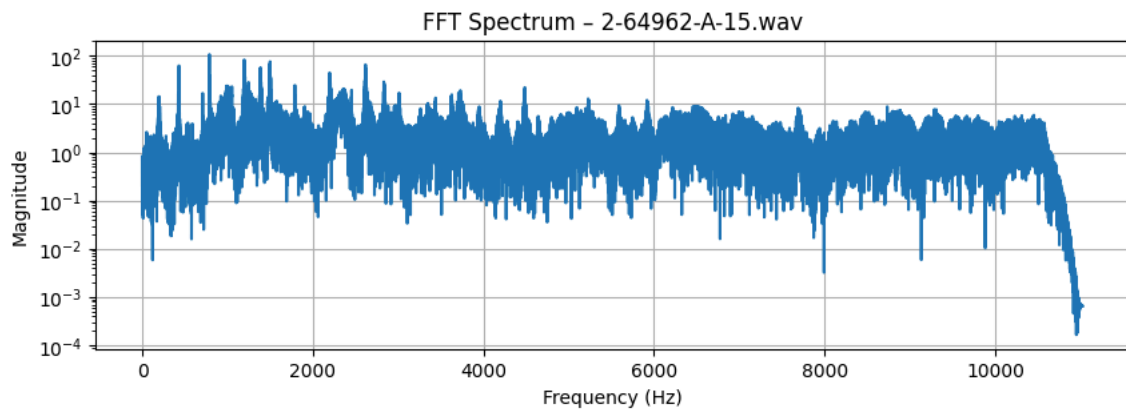
```
# -----
results.append({
    "File": name,
    "Detected_noise_Hz": noise_freq,
    "Correlation_IIR": corr_iir,
    "Correlation_FIR": corr_fir
})

# -----
# SUMMARY TABLE
# -----
df = pd.DataFrame(results)
print("\n=====")
print(" FINAL SUMMARY TABLE")
print("=====")
display(df)
```


=====

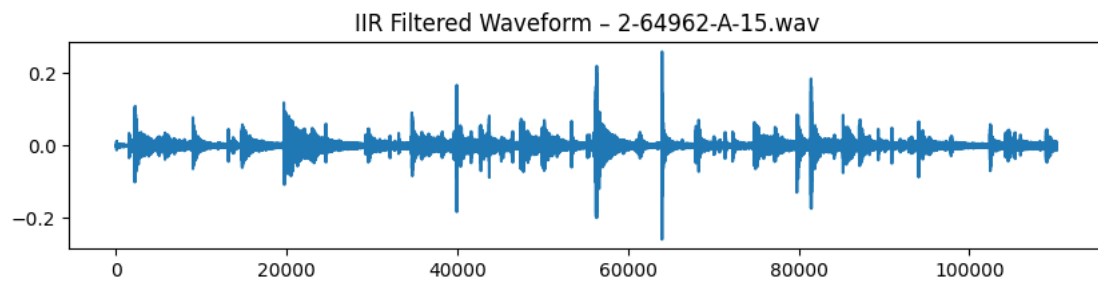
Q4 – IIR & FIR RESULTS BELOW

=====

Processing: 2-64962-A-15.wav
-----

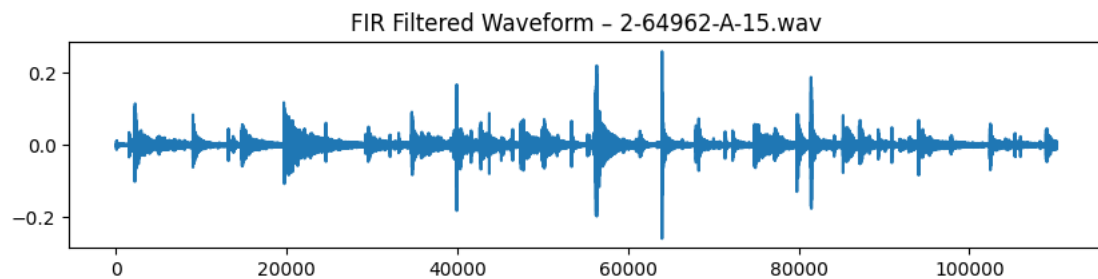
Detected Noise Frequency: 1196.8 Hz
IIR Filter Output:

0:00 / 0:05

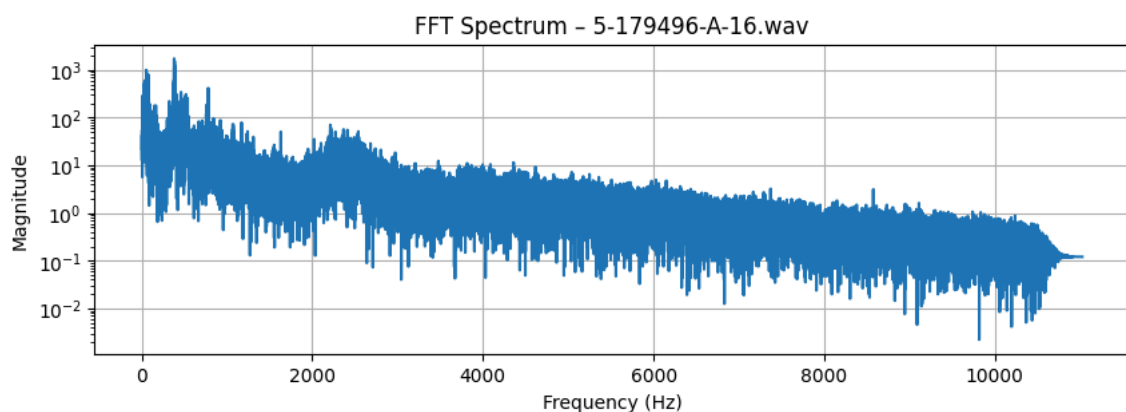


Correlation (Clean vs IIR): -0.004103419929829594
FIR Filter Output:

0:00 / 0:05



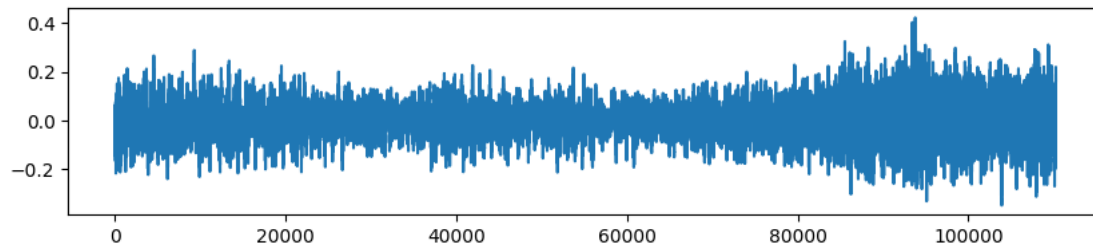
Correlation (Clean vs FIR): -0.0021836937361392646

Processing: 5-179496-A-16.wav
-----

Detected Noise Frequency: 380.0 Hz
IIR Filter Output:

0:00 / 0:05

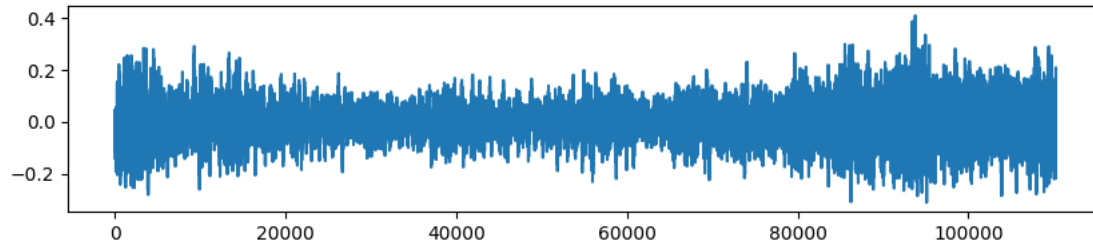
IIR Filtered Waveform - 5-179496-A-16.wav



Correlation (Clean vs IIR): -0.009888348144168094
FIR Filter Output:

0:00 / 0:05

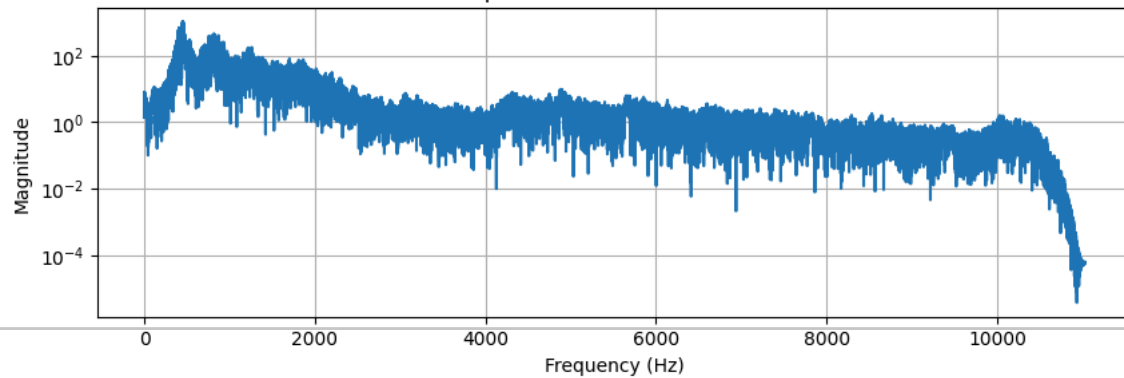
FIR Filtered Waveform - 5-179496-A-16.wav



Correlation (Clean vs FIR): -0.0064741046679618166

Processing: 2-114280-A-0.wav

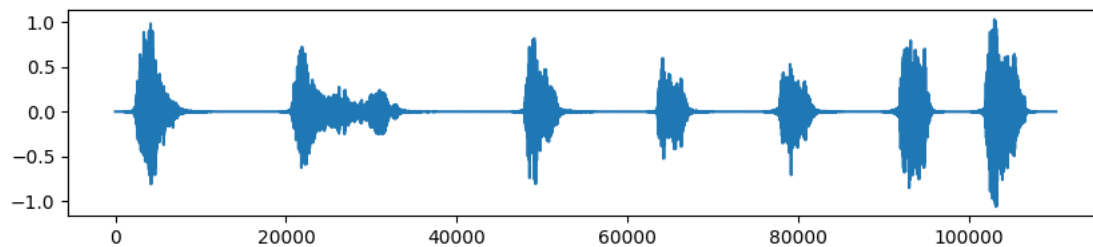
FFT Spectrum - 2-114280-A-0.wav



Detected Noise Frequency: 449.6 Hz
IIR Filter Output:

0:00 / 0:05

IIR Filtered Waveform - 2-114280-A-0.wav



Correlation (Clean vs IIR): 0.006253667399250376
FIR Filter Output: